

FasalDoc — Errors & Pitfalls Across the Project Lifetime

A reference guide to what commonly breaks, and how to catch it early

Organized by phase. Bookmark this — most teams hit 60-70% of this list at some point; knowing it in advance turns a panic into a checklist.

Phase 1: Data Collection & Preparation

Issue	Why it happens	How to catch/prevent it
Class imbalance	Some diseases are rare/hard to photograph, so you end up with 500 "healthy" images and 20 "blight" images	Check class counts before training (<code>len(os.listdir(folder))</code> per class); use class weighting in training or targeted collection for underrepresented classes
Label noise/mislabeling	Farmer guesses or rushed labeling during field trips are often wrong	Get every label reviewed by your agronomist/vet advisor before it enters the training set; never train directly on unverified crowd-sourced labels

Data leakage between train/val/test	Multiple photos of the <i>same</i> diseased leaf end up split across train and validation, inflating accuracy artificially	Split by <i>plant/sample</i> , not by individual image, when you have multiple photos of the same specimen
Inconsistent image quality across sources	PlantVillage (clean lab photos) mixed with your local phone photos (varied lighting/blur) confuses the model	Use data augmentation aggressively; consider training separate "clean" and "field" evaluation sets to track real-world performance specifically
Missing/corrupted files after unzip	Kaggle downloads or zip transfers sometimes partially fail	Always run a quick integrity check (<code>PIL.Image.open()</code> on every file in a try/except) before training — corrupted files crash training mid-run, often confusingly

Phase 2: Model Training

Issue	Why it happens	How to catch/prevent it
High train accuracy, low validation accuracy (overfitting)	Too few images per class, or model memorizing lab-condition backgrounds instead of disease features	Watch the gap between train/val accuracy every epoch; add dropout, augmentation, or more data if the gap grows

Model "cheating" on background instead of the leaf	PlantVillage images often have identical clean backgrounds per class — model learns to recognize the <i>background</i> , not the disease	This is a well-documented PlantVillage pitfall — always validate on PlantDoc (real backgrounds) before trusting accuracy numbers
Colab session disconnects mid-training	Free Colab has usage limits and idle timeouts	Save checkpoints periodically (<code>ModelCheckpoint</code> callback) so you don't lose hours of training; consider Colab Pro if this becomes a recurring blocker
Out-of-memory errors on GPU	Batch size too large for available GPU memory	Reduce <code>BATCH_SIZE</code> (try 16 instead of 32); this is usually the fastest fix
Accuracy looks great but confidence scores are all near 100%	Model overconfidence, common with softmax + limited data	Don't fully trust raw confidence numbers early on; consider temperature scaling or just keep your <code>CONFIDENCE_THRESHOLD</code> conservative until validated against real users
TFLite conversion changes model behavior slightly	Quantization during conversion can shift predictions marginally	Always re-test the <code>.tflite</code> version's accuracy separately from the original <code>.keras</code> model — don't assume they perform identically

Phase 3: Backend (FastAPI)

Issue	Why it happens	How to catch/prevent it
"Model not loaded" errors on startup	Model file path wrong, or file missing when container/server starts	Check the <code>/health</code> endpoint immediately after any deploy — it's built specifically to catch this
Image preprocessing mismatch between training and serving	If training used different <code>resize/normalization</code> than the backend's <code>preprocess_image()</code> , predictions will be silently wrong (not crash — just inaccurate)	Keep preprocessing logic identical between <code>train_baseline_model.py</code> and <code>main.py</code> — consider extracting it into one shared function used by both
CORS errors when mobile app calls the API	Default browser/app security blocking cross-origin requests	Already handled in the skeleton with permissive CORS for development — remember to restrict <code>allow_origins</code> before any public launch
Large image uploads timing out or failing	Phone cameras produce large files (several MB); slow networks or default timeouts choke on this	Compress images client-side before upload (already done via <code>quality: 0.7</code> in the capture screen) — increase server timeout settings if still an issue
"Class index out of range" errors	<code>class_names.json</code> doesn't match the number of output classes the model was actually trained on	Regenerate <code>class_names.json</code> fresh every time you retrain — a stale file from an old training run is a very common silent bug
API works locally but fails on physical phone	Using <code>localhost</code> instead of your machine's actual local network IP	Always double check <code>API_URL</code> uses the real local IP address when testing on a device, not <code>localhost</code> or <code>127.0.0.1</code>

Phase 4: Mobile App

Issue	Why it happens	How to catch/prevent it
Camera permission denied silently	User denies permission, app doesn't handle it gracefully	The skeleton already shows a permission request screen — test the "deny" path explicitly, not just the "allow" path
App works in Expo Go but breaks in a real build	Some native modules behave differently between Expo Go and a standalone build	Test an actual EAS build (<code>eas build</code>) well before any demo day, don't assume Expo Go behavior is final
AsyncStorage data disappears	User reinstalls the app, or storage limits hit on low-end devices	Fine for MVP/pilot, but communicate this limitation to pilot users; plan backend-synced history for Phase 2
Slow performance on low-end Android devices	Many rural users will have budget phones, not flagship devices	Test on an actual low-end/older Android device, not just your own phone — this is one of the most commonly skipped steps by student teams
Image orientation flips (sideways photos)	Some Android camera APIs don't auto-correct EXIF orientation	Test photos taken in both portrait and landscape; add EXIF-based rotation correction if you see this

Phase 5: DevOps & Deployment

Issue	Why it happens	How to catch/prevent it
Docker build works locally, fails in CI	Different base image/ architecture (e.g. Apple Silicon Mac vs. Linux CI runner)	Specify platform explicitly if needed (<code>--platform linux/amd64</code>); always test the exact CI build command locally before pushing
Image too large / slow to build	TensorFlow images are notoriously large	Consider <code>tensorflow-cpu</code> instead of full <code>tensorflow</code> if you don't need GPU in production serving; use <code>.dockerignore</code> properly (already set up)
Secrets accidentally committed to GitHub	Hardcoded API keys or credentials in code	Use environment variables + GitHub Secrets from day one, even for MVP; add a <code>.env</code> to <code>.gitignore</code> immediately
Free-tier cloud hosting cold starts	Services like Cloud Run scale to zero, first request after idle is slow	Acceptable for MVP demo but mention it if judges test the live app after a pause — or "warm" it right before your demo slot
Team overwrites each other's work	No branching strategy, direct pushes to <code>main</code>	Adopt a simple branch + PR workflow now, even for a small team — saves real pain later

Phase 6: Project/Team-Level Risks (non-technical, but just as common)

Issue	Why it happens	How to catch/prevent it
-------	----------------	-------------------------

Scope creep	Excitement leads to adding livestock/soil/marketplace features before crop detection even works well	Refer back to your phased roadmap — resist adding Phase 2/3 features until Phase 1 is solid and demoed
One person becomes a bottleneck	Usually the ML lead, since everything downstream depends on the model	Get a <i>rough</i> working model early (even mediocre accuracy) so backend/mobile teammates aren't blocked waiting for a "perfect" model
No one owns the remedy database accuracy	Feels like "just data entry," gets deprioritized	Treat this as seriously as the ML model — wrong remedy advice is your single biggest real-world liability risk
Losing momentum after HATCH submission	Adrenaline of a deadline disappears afterward	Set your next milestone (pilot expansion, funding conversation) <i>before</i> demo day, not after
Team disagreement on scope/direction	Natural in any 5-person team under deadline pressure	Weekly sync (already in your build plan) is there specifically to surface disagreements early, not let them fester

The One Habit That Prevents Most of This

Test the full end-to-end loop early and often — camera → backend → prediction → remedy displayed — not just each piece in isolation. Most of the errors above are individually easy to fix, but they hide well when each teammate only tests their own piece. A working full-loop test

once a week, from Month 1 onward, will surface almost everything on this list while it's still cheap to fix.